Inside the parenthesis of the function are two values separated by a comma. The first value is the number of the pin that we are setting up. This could be a number or variable with the value ranging from 0 to 13 or A0 to A5 (if using the Analog In pins for digital I/O) corresponding to the pin number printed on the interface board.

The second value is the state that we need for the pin to work with our circuit and can include either of two predefined constants: `INPUT` or `OUTPUT`. Using `INPUT` in a `pinMode()` function will place the pin in a state of high impedance, where the pin is configured to receive an incoming signal yet places very little load on the overall circuit. This is good for reading sensitive inputs without affecting the sensor in any way. A digital input pin is only sensitive to two values, `HIGH` and `LOW`.

By using `OUTPUT`, the digital pin is placed in a state of low impedance and can, therefore, source or sink a respectable current, considering the size of such a little chip, of 40 milliamps to other circuits. This is enough current to brightly light up an LED while providing little resistance to the rest of the circuit.

In this project, we use `pinMode()` two different ways:

```
pinMode(speakerPin, OUTPUT);
pinMode(sensorPin, INPUT);
```

The first line sets the speaker pin to an `OUPUT` state so that we can connect and power a small external speaker. The second line establishes the sensor pin as an `INPUT`, making it ready to receive signals from a low power sensor or other input.

For every digital I/O pin that we need to use, we must first establish its mode once in the program, usually done in the `setup()` function. This can get a bit tedious if we plan to use all 14 digital pins as outputs. Instead, we can use a `for` loop as shown in previous examples to set up each pin as an output, like in the following example:

```
for (int i=0; i<14; i++) pinMode(i, OUTPUT);
```

This line is a simple `for` statement used to repeat a line of code a total of 14 times, placing each digital pin into an `OUTPUT` mode. This could come in handy and save you some typing if you're trying to hook up 14 LEDs to one Arduino.

digitalWrite()

Once we have a pin established as an `OUTPUT`, it is then possible to turn that pin on or off using the `digitalWrite()` function. Its syntax follows:

```
digitalWrite(pin, state)
```

There are two statements that are used with this function that include pin number and state. Just like `pinMode()`, this could be a number or variable with the value ranging from 0 to 13 or A0 to A5. The second statement is the state of the output pin that corresponds to the two predefined constants: `HIGH` and `LOW`.

`HIGH` is the state of sourcing current and provides a connection to +5 VDC. `LOW`, the default of any output pin, is the state of sinking current, providing a connection to ground. If the circuit is configured similar to Figure 5-2, where the output pin is configured to source current for a circuit, setting the pin `HIGH` is basically turning the circuit on, while setting the pin `LOW` turns it off. This would be reversed if the circuit is more like Figure 5-3 and the output pin needs to sink current in order to turn on the circuit. This is why in this book we will generally stick to sourcing current in our circuits so that `HIGH` will most likely mean "turn something on."

Just like we did with the `pinMode()` function, it is possible to turn on or off all of the pins using a `for` loop. Take the following code sample: